Validation of SLT Parameter Estimation for RelaLeap
A Mathematical and Software Plan for WBIC/SGLD/LLC Diagnostics

GPT-5.5-Pro for Ben Goertzel

July 4, 2026

Abstract

This document gives a systematic plan for validating finite-sample Singular Learning Theory (SLT) parameter estimators for RelaLeap. It is intended for human project leads and for coding agents implementing the validation package. The main estimators are WBIC/SGLD-based finite-sample proxies for the local learning coefficient (LLC), also called the real log canonical threshold (RLCT), together with block-wise LLC interaction information. The document emphasizes that these are calibrated diagnostics, not exact RLCTs and not causal certificates. The validation program must show that the estimator recovers known analytic targets, behaves correctly under sample-size scaling, handles gauge symmetries and block interactions, and works on RelaLeap-shaped singular benchmark families before it is used to guide decisions about residual-column factorizations. The final sections give a parallelized plan for multiple coding subagents.

Contents

1 Purpose and context

RelaLeap aims to determine whether sparse columnar residual adapters on top of a frozen sequence model can learn reusable, causally separable residual corrections. The central scientific question is not merely whether a residual adapter lowers cross-entropy or reconstructs a dense teacher residual. The question is whether a proposed factorization has good local evidence geometry while also passing causal-modularity gates: support regret, off-support leakage, finite-update commutators, exact ablation checks, and matched null controls.

SLT is relevant because it provides a way to talk about the local evidence volume around a fitted solution. In singular models, the ordinary parameter count penalty is wrong. The leading evidence penalty is governed by a local learning coefficient

$$\lambda,$$

which behaves like an effective dimension of the fitted stratum. RelaLeap would like to use estimates of such quantities to compare residual factorizations and to diagnose whether modules are independent, redundant, or synergistic.

However, the project must avoid a common failure mode: producing a number that looks like an SLT quantity without validating that the number estimates the intended object. A previous seven-arm smoke harness was useful as engineering scaffolding, but it used tiny toy caches, a handcrafted complexity proxy, and a small WBIC-like run over frozen output-amplitude multipliers rather than the true adapter parameters. Such a result can test report plumbing, but it cannot establish an SLT claim. The validation plan below is designed to prevent this failure mode.

1.1 The prior implementation guide

A prior implementation note already specified useful foundations: notation for loss and total energy, SGLD update rules, WBIC temperature conventions, block-wise LLC interaction information, calibration benchmarks, diagnostics, and report schemas. This document assumes no knowledge of that note, but it builds on its core rule:

> The output of the SLT estimator package is a calibrated diagnostic panel. It is not a certificate that a discovered column is causal. Promotion still requires causal audits, null controls, bootstrap uncertainty, and finite-sample caveats.

The goal here is to sharpen and extend the validation side of that note.

2 Estimator target and notation

Let

$$D_n = \{(x_i, y_i)\}_{i=1}^n$$

be an estimation set. Let

$$\theta \in \Theta \subseteq \mathbb{R}^p$$

be the parameter block being sampled. In RelaLeap, θ should be a live adapter block such as atoms, dictionary parameters, router parameters, membership parameters, shared-core parameters, or a declared union of such blocks. The frozen base transformer is used to evaluate the loss, but its parameters are not part of θ.

Let

$$\ell_i(\theta)$$

be the per-example negative log-likelihood, cross-entropy, or residual Gaussian negative log-likelihood. Define the mean loss and total energy by

$$L_n(\theta) = \frac{1}{n}\sum_{i=1}^n \ell_i(\theta), \qquad E_n(\theta) = nL_n(\theta) = \sum_{i=1}^n \ell_i(\theta).$$

Let

$$U(\theta) = -\log \pi(\theta)$$

be the negative log prior, and let $\hat{\theta}$ be a trained reference point, ideally a local MAP for the same loss-plus-prior objective used during sampling.

2.1 Tempered local posterior

The local tempered target is

$$p_\beta(\theta \mid D_n) \propto \exp\{-\beta E_n(\theta) - U(\theta)\}$$

possibly restricted or softly localized to a neighborhood of $\hat{\theta}$. For WBIC, the intended total-energy convention is

$$\beta_{\mathrm{WBIC}} = \frac{1}{\log n},$$

so the target is

$$p(\theta) \propto \exp\left\{-\frac{E_n(\theta)}{\log n} - U(\theta)\right\}.$$

If the implementation uses mean loss internally, the coefficient applied to the mean loss must be

$$\frac{n}{\log n}.$$

Equivalently, if an API stores a per-example coefficient, it may record

$$\frac{1}{n \log n},$$

but the report must also explicitly record the effective coefficient on total energy. A code path that multiplies total energy by $1/(n \log n)$ instead of $1/\log n$ is too hot by a factor of n and must fail validation.

2.2 SGLD update

For

$$\Phi(\theta) = \beta E_n(\theta) + U(\theta),$$

plain SGLD uses

$$\theta_{t+1} = \theta_t - \frac{\eta_t}{2}\nabla_\theta \Phi(\theta_t) + \sqrt{\eta_t}\xi_t, \qquad \xi_t \sim \mathcal{N}(0, I).$$

With a minibatch B_t of size m,

$$\nabla E_n(\theta) \approx \frac{n}{m}\sum_{i \in B_t} \nabla \ell_i(\theta).$$

For small calibration problems, full-batch gradients should be mandatory. For larger RelaLeap adapter blocks, minibatch gradients may be used only with a minibatch-noise diagnostic.

2.3 WBIC and LLC proxy

In singular models, local free energy has the asymptotic form

$$F_n = nL_n(\theta^*) + \lambda \log n - (m-1)\log\log n + O_p(1),$$

where λ is the LLC/RLCT and m is the multiplicity. A finite-sample WBIC-style LLC proxy is

$$\hat{\lambda}_n = \frac{\mathbb{E}_{\beta_{\mathrm{WBIC}}}[E_n(\theta)] - E_n(\hat{\theta})}{\log n}.$$

This value is not the exact RLCT. It depends on sample size, prior, parameterization, local-basin definition, optimization quality, sampler mixing, and the omitted multiplicity term.

2.4 LLC interaction information

For blocks A and B, define the block-wise interaction proxy

$$I_\lambda(A;B) = \hat{\lambda}_A + \hat{\lambda}_B - \hat{\lambda}_{A\cup B}.$$

Its intended interpretation is:

$$I_\lambda(A;B) \approx 0 \quad \text{independent or additive local evidence,}$$

$$I_\lambda(A;B) > 0 \quad \text{redundant or shared singular structure,}$$

$$I_\lambda(A;B) < 0 \quad \text{synergistic or emergent joint structure.}$$

This sign convention must be validated on analytic and RelaLeap-shaped benchmarks before it is trusted on real residual adapters.

3 Validation principles

Principle 1 (Validate the target, not only the sampler)**.** *A stable SGLD energy trace is not enough. The validation suite must show that the estimator recovers known analytic LLC values, known interaction signs, and known sample-size trends.*

Principle 2 (Actual parameters only)**.** *The sampled block must contain the actual trainable parameters of the residual arm or a declared sub-block of it. Sampling frozen scalar output multipliers is allowed only as a smoke test and must not be reported as SLT evidence.*

Principle 3 (Known analytic targets first)**.** *Every calibration benchmark should specify*

$$K(\theta), \quad \lambda_{\text{true}}, \quad m_{\text{true}}, \quad \lambda_A, \quad \lambda_B, \quad \lambda_{AB}, \quad I_\lambda(A;B)$$

whenever these quantities are known. Qualitative checks such as "less than parameter count over two" are useful but insufficient.

Principle 4 (Sample-size scaling is mandatory)**.** *A single n can be dominated by finite-sample constants. Run*

$$n \in \{256, 512, 1024, 2048, 4096, 8192\}$$

and fit

$$\Delta E_n = a \log n + b \log\log n + c,$$

where

$$\Delta E_n = \mathbb{E}_\beta[E_n(\theta)] - E_n(\hat{\theta}).$$

Report both single-n and slope-fit estimates.

Principle 5 (Do not clip negative estimates)**.** *If $\hat{\lambda} < 0$, do not clip it to zero. A negative value usually means that $\hat{\theta}$ is not the local MAP for the sampled objective, the sampler target is wrong, the energy is evaluated inconsistently, or Monte Carlo error is large. Negative values should cause a failed or diagnostic-only status.*

Principle 6 (Gauge symmetries must be controlled)**.** *Rank-one atoms and nonorthogonal dictionaries have scale, sign, permutation, and rotation-like symmetries. Without a gauge policy, the estimator may measure parameterization artifacts rather than functional degeneracy.*

Principle 7 (Block-wise interactions need full-adapter checks)**.** *Block-wise I_λ estimates answer a conditional question with other blocks frozen. For important claims, also run versions with shared cores sampled and at least one full-adapter sampling pass with projections.*

Principle 8 (Lower LLC is not automatically better)**.** *A dead, collapsed, underfit, or overly constrained adapter can have low apparent LLC. Task/reconstruction gates, null controls, causal audits, commutator budgets, and leakage budgets must be passed before WBIC/LLC is used for model selection.*

4 Calibration benchmark suite

This section defines the benchmark families the estimator must pass before any RelaLeap residual-arm SLT number is treated as evidence.

4.1 Regular identifiable linear Gaussian model

Data:

$$x_i \sim \mathcal{N}(0, I_d), \qquad y_i = x_i^T w^* + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Parameter $w \in \mathbb{R}^d$. Known target:

$$\lambda = \frac{d}{2}, \qquad m = 1.$$

This verifies temperature scaling, Gaussian prior handling, total-energy accounting, and the WBIC formula in the easiest case.

4.2 Rank-deficient linear regression

Let

$$y_i = z_i^T u^* + \epsilon_i, \qquad x_i = Rz_i,$$

where $R \in \mathbb{R}^{d\times r}$ has rank $r < d$. Fit $w \in \mathbb{R}^d$ with prediction $x_i^T w$. The effective identifiable contribution should be near

$$\lambda \approx \frac{r}{2},$$

not $d/2$. This catches estimators that merely return parameter count over two.

4.3 Exact Gaussian posterior test

Use a pure quadratic energy

$$L(\theta) = \frac{1}{2}(\theta - \hat{\theta})^T H (\theta - \hat{\theta})$$

with Gaussian prior. The tempered posterior is known exactly:

$$p_\beta(\theta) \propto \exp\left[-\frac{1}{2}\theta^T(\beta H + \Sigma_0^{-1})\theta\right].$$

Test sample mean, sample covariance, energy mean, and $\hat{\lambda}$. This catches errors in noise scale, preconditioning, prior gradients, and energy scaling.

4.4 Product singularity with negative synergy

Use

$$K(a, b) = (ab)^2$$

at the optimum $(a, b) = (0, 0)$. If block $A = a$ is sampled with $b = 0$ frozen, movement in a alone does not change the function; likewise for $B = b$. But joint movement creates loss variation. The expected qualitative pattern is

$$\lambda_A \approx 0, \qquad \lambda_B \approx 0, \qquad \lambda_{AB} > 0, \qquad I_\lambda(A; B) < 0.$$

This is the cleanest basic benchmark for emergent joint structure.

4.5 Nonzero product ridge with redundancy

Use a local form such as

$$K(a, b) = (ab - c)^2, \qquad c \neq 0,$$

near a solution with $a \neq 0$, $b \neq 0$. The joint solution has a ridge: changing a and b together can preserve the product. Freezing one block makes the other locally regular. A common expected pattern is

$$\lambda_A \approx \frac{1}{2}, \qquad \lambda_B \approx \frac{1}{2}, \qquad \lambda_{AB} \approx \frac{1}{2}, \qquad I_\lambda(A; B) > 0.$$

This verifies that the estimator distinguishes redundancy from synergy.

4.6 Cusp and crossing singularities

Include small analytic losses such as

$$K(a, b) = (a^2 - b^2)^2$$

and

$$K(a, b, c) = (ab - c)^2 + c^2.$$

These expose non-isolated minima, ridges, multiplicity effects, and singular strata that are more representative of neural parameterizations than regular quadratics.

4.7 Overparameterized mixture

Fit a two-component one-dimensional Gaussian mixture to data generated by a single Gaussian component:

$$p(y \mid \theta) = \alpha \mathcal{N}(y; \mu_1, \sigma^2) + (1 - \alpha)\mathcal{N}(y; \mu_2, \sigma^2).$$

The model is singular due to permutation symmetry and component redundancy. The estimator should detect

$$\lambda < p/2$$

for $p = 3$, but this benchmark alone is not enough because the exact target is more delicate.

4.8 Composition benchmarks

Define three composition families.

Independent composition.

$$y_i = f_A(x_{iA}; \theta_A) + f_B(x_{iB}; \theta_B) + \epsilon_i$$

with independent inputs and disjoint parameters. Expect

$$\lambda_{AB} \approx \lambda_A + \lambda_B, \qquad I_\lambda(A; B) \approx 0.$$

Redundant composition. Two blocks share a latent direction or scalar:

$$r_A(h) = \alpha_A us(h), \qquad r_B(h) = \alpha_B us(h).$$

Expect

$$I_\lambda(A; B) > 0.$$

Synergistic composition. Use a generator where neither block alone can express the mechanism, for example the zero-product benchmark or a gated pair. Expect

$$I_\lambda(A; B) < 0$$

under the pre-registered block-conditioning protocol.

4.9 RelaLeap-shaped benchmark suite

Generic SLT benchmarks are not enough. The estimator must also pass benchmarks shaped like the actual RelaLeap architecture.

Rank-one atom gauge benchmark. Use

$$A_g(h) = s_g b_g(a_g^T h)$$

with normalized and unnormalized parameterizations. Check that gauge fixing changes parameter coordinates without materially changing the functional LLC estimate.

Dead-column benchmark. Create a column with zero router probability or zero amplitude. The estimator should report low effective contribution, but the decision logic must not treat this as modular evidence. Causal audits should identify router collapse or dead structure.

Shared-core benchmark. Two columns share a core direction:

$$r_A(h) = \alpha_A us(h), \qquad r_B(h) = \alpha_B us(h).$$

Expected interaction:

$$I_\lambda(A; B) > 0.$$

Mediator benchmark. Columns A and B are coupled through a mediator M. Pairwise $I_\lambda(A; B)$ may look strong without conditioning, but should weaken after sampling or conditioning on M. This tests whether the estimator can support mediator-column interpretations rather than dense all-to-all coupling.

Router-collapse benchmark. Multiple columns exist, but the router selects one column nearly always. The estimator should not reward low apparent complexity as modular success.

Low-rank trap benchmark. The true generator is a dense low-rank residual adapter. SVD or low-rank controls should win, and columnar claims should be blocked.

Oblique dictionary benchmark. The true factors are nonorthogonal. A nonorthogonal dictionary should beat an orthogonal sparse basis, while causal and SLT diagnostics should show whether it is a real factorization or merely a better compressor.

5 Implementation requirements

5.1 Module layout

A recommended package layout is:

```
relaleap/slt/__init__.py
relaleap/slt/parameter_blocks.py
relaleap/slt/energy.py
relaleap/slt/priors.py
relaleap/slt/sgld.py
relaleap/slt/wbic.py
relaleap/slt/llc_interactions.py
relaleap/slt/analytic_registry.py
relaleap/slt/calibration_benchmarks.py
relaleap/slt/sample_size_sweep.py
relaleap/slt/gauge.py
relaleap/slt/nulls.py
relaleap/slt/prior_sensitivity.py
relaleap/slt/minibatch_noise.py
relaleap/slt/reports.py
relaleap/slt/ci_gates.py
```

5.2 SGLD pseudocode

Trust-region shrinkage or proposal rejection may be useful for engineering stability probes, but such runs should not be used as calibration evidence unless the stationary distribution of the modified sampler is explicitly characterized.

5.3 MAP reference validation

Before running WBIC, refine $\hat{\theta}$ on the sampled block with all other blocks frozen, using the same loss and prior as the sampler. Record:

```
grad_norm_at_theta_hat
map_refinement_steps
energy_before_refinement
energy_after_refinement
loss_decrease_after_refinement
theta_hat_status = local_map_pass | local_map_fail
```

Algorithm 1 SGLD for a declared parameter block

Require: Loss closure L_n, sampled parameters θ, reference $\hat{\theta}$, prior U, configuration cfg

Set random seed from $cfg.seed$
Initialize $\theta \leftarrow \hat{\theta} + \sigma_{init}\epsilon$
Initialize empty energy trace and sample buffer
for $t = 1$ to $cfg.num\_steps$ **do**
    Select batch B_t or full data
    Compute mean loss $L_B(\theta)$
    Estimate total-energy gradient

$$g_E \leftarrow \frac{n}{|B_t|} \sum_{i \in B_t} \nabla_\theta \ell_i(\theta)$$

    Compute prior gradient $g_U \leftarrow \nabla_\theta U(\theta)$
    Set $g \leftarrow \beta g_E + g_U$
    Set step size η_t
    Draw $\xi_t \sim \mathcal{N}(0, I)$
    Update

$$\theta \leftarrow \theta - \frac{\eta_t}{2} g + \sqrt{\eta_t}\xi_t$$

    if energy evaluation interval **then**
        Evaluate $E_n(\theta)$ on fixed full data or deterministic probe
        Record energy and diagnostics
    end if
    if post-burn-in and thinning condition **then**
        Save sampled parameter block or sufficient statistics
    end if
end for
return samples, energy trace, diagnostics

If the deterministic refinement materially lowers energy, the original $\hat{\theta}$ is not a valid reference for the estimator.

5.4 Gauge policy

For each arm type, define a gauge policy:

```
gauge_policy:
  normalize_read_vectors: true
  normalize_write_vectors: true
  absorb_scale_into_amplitude: true
  canonical_sign: first_nonzero_positive
  sort_atoms_by_usage_or_norm: true
  report_permutation_symmetry: true
```

Also implement a test that function-preserving gauge transformations do not materially change the functional prediction or the calibrated LLC estimate.

5.5 Report schema

Every estimator run should output:

```
{
  "estimator_status": "calibrated|diagnostic_only|failed",
  "estimator_kind": "finite_sample_wbic_sgld_proxy_not_exact_rlct",
  "energy_definition": "total_nll|mean_loss",
  "coefficient_on_total_nll": float,
  "n_data": int,
  "beta_total_energy": float,
  "lambda_single_n": float,
  "lambda_slope_fit": float | null,
  "multiplicity_proxy_b": float | null,
  "lambda_stderr": float,
  "lambda_ci95": [float, float],
  "num_chains": int,
  "num_steps": int,
  "burn_in": int,
  "thin": int,
  "energy_ess_per_chain": list,
  "rhat_energy": float,
  "trend_test": dict,
  "nan_or_divergence_count": int,
  "negative_lambda_policy": "fail_or_diagnostic_never_clip",
  "prior_family": string,
  "prior_scale": float,
  "gauge_policy_id": string,
  "theta_hat_status": string,
  "calibration_suite_status": string
}
```

6 Decision gates

Gate 1 (Sampler correctness)**.** *The exact Gaussian posterior test must pass for mean, covariance, energy mean, and λ under both total-energy and mean-loss conventions.*

Gate 2 (Analytic calibration)**.** *Regular linear, rank-deficient linear, product-singular, independent, redundant, and synergistic benchmarks must pass their pre-registered target signs and magnitudes.*

Gate 3 (Sample-size scaling)**.** *For benchmarks with known λ, the slope-fit estimate from $\Delta E_n = a \log n + b \log \log n + c$ must agree with the analytic value within pre-registered tolerance.*

Gate 4 (Gauge stability)**.** *Function-preserving gauge changes must not materially change predictions or functional LLC estimates. If they do, the parameterization is not controlled.*

Gate 5 (Interaction sanity)**.** *Block-wise, shared-core-conditioned, and full-adapter projected interaction protocols must agree qualitatively on calibration cases. If block-wise I_λ suggests sparse structure but full-adapter sampling shows diffuse all-to-all interaction, the columnar interpretation is blocked.*

Gate 6 (Null normalization)**.** *Estimator-specific nulls must be run. These include target shuffles, residual shuffles, block-assignment shuffles, router-support shuffles, shared-core shuffles, and gauge randomization. A real interaction claim must exceed its null distribution with uncertainty.*

Gate 7 (RelaLeap promotion)**.** *No RelaLeap residual arm may be promoted because of low $\hat{\lambda}$ alone. Promotion requires ordinary task/reconstruction controls, dependency-aware nulls, causal audits, commutator/leakage/support-regret budgets, and only then SLT evidence among surviving candidates.*

7 Parallel coding-subagent plan

The validation package can be accelerated substantially by running multiple coding subagents in parallel. The key is to define stable interfaces first, then let agents implement independent benchmark families, diagnostics, and reports against those interfaces. The work should not be fully sequential. Only the shared contracts and minimal SGLD core are truly blocking.

7.1 Shared contracts for all subagents

Before parallel work begins, create a small shared interface package:

```
class EnergyModel:
    def total_energy(theta, data): ...
    def mean_loss(theta, batch): ...
    def grad_total_energy(theta, batch): ...
    def theta_hat(self): ...
    def parameter_blocks(self): ...

class CalibrationBenchmark:
    benchmark_id: str
    def make_data(n, seed): ...
    def make_energy_model(data, cfg): ...
    def true_targets(self): ...
```

```
class EstimatorResult:
    lambda_hat: float
    lambda_se: float
    status: str
    diagnostics: dict
```

Each subagent must write tests and a small example command. Each branch must produce machine-readable artifacts and a short `decision.md`.

7.2 Subagent A: SGLD and WBIC core

Mission. Implement the core SGLD sampler, WBIC estimator, energy accounting, chain diagnostics, and exact Gaussian posterior test hooks.

Deliverables.

```
relaleap/slt/sgld.py
relaleap/slt/wbic.py
relaleap/slt/energy.py
relaleap/slt/diagnostics.py
tests/test_sgld_gaussian.py
tests/test_temperature_conventions.py
```

Key tests.

1. Total-energy and mean-loss conventions yield the same effective target.
2. Total NLL multiplied by $1/(n \log n)$ fails the negative test.
3. Exact Gaussian posterior mean and covariance are recovered.
4. Energy ESS, split-chain $\hat{R}$, trend, and NaN diagnostics are reported.
5. Negative $\hat{\lambda}$ is not clipped.

Dependencies. Needs only the shared interfaces. This is a wave-0 or wave-1 blocking subagent.

7.3 Subagent B: analytic calibration registry

Mission. Implement a registry of small analytic SLT benchmarks with pre-registered target values and signs.

Deliverables.

```
relaleap/slt/analytic_registry.py
relaleap/slt/calibration_benchmarks.py
tests/test_regular_linear_calibration.py
tests/test_rank_deficient_calibration.py
tests/test_product_singularity.py
tests/test_redundant_product_ridge.py
tests/test_composition_interactions.py
```

Benchmarks.

1. Regular linear Gaussian, $\lambda = d/2$.
2. Rank-deficient linear, $\lambda \approx r/2$.

3. Product singularity $K = (ab)^2$, expected negative interaction.

4. Nonzero product ridge $K = (ab - c)^2$, expected positive interaction.

5. Independent composition, expected $I_\lambda \approx 0$.

6. Redundant shared-core composition, expected $I_\lambda > 0$.

7. Synergistic composition, expected $I_\lambda < 0$.

Dependencies. Can start in parallel with Subagent A using mock estimator outputs, then integrate once the sampler exists.

7.4 Subagent C: sample-size and multiplicity sweep

Mission. Implement n-sweeps and slope fitting for ΔE_n.

Deliverables.

```
relaleap/slt/sample_size_sweep.py
relaleap/slt/multiplicity_proxy.py
tests/test_slope_fit_regular_linear.py
tests/test_slope_fit_product_singularity.py
```

Required output.

```
n_values: [256, 512, 1024, 2048, 4096, 8192]
lambda_single_n
lambda_slope_fit
multiplicity_proxy_b
fit_residuals
sample_size_sensitivity_status
```

Dependencies. Needs Subagent A for real runs, but can write code and unit tests with synthetic traces in parallel.

7.5 Subagent D: MAP reference and prior handling

Mission. Ensure $\hat{\theta}$ is a valid local MAP reference and priors are consistently applied.

Deliverables.

```
relaleap/slt/priors.py
relaleap/slt/map_refinement.py
relaleap/slt/prior_sensitivity.py
tests/test_prior_gradients.py
tests/test_map_refinement_gate.py
tests/test_prior_sensitivity_report.py
```

Key tests.

1. Prior gradients match finite differences.

2. MAP refinement does not materially lower energy on calibrated cases.

3. Prior-scale sweeps report ranking stability or instability.

4. The report says diagnostic-only if arm rankings flip under modest prior changes.

Dependencies. Parallel after shared energy interface exists.

7.6 Subagent E: gauge canonicalization

Mission. Implement gauge policies for rank-one atoms and dictionaries.

Deliverables.

```
relaleap/slt/gauge.py
relaleap/slt/releap_parameterizations.py
tests/test_rank_one_gauge_invariance.py
tests/test_dictionary_scale_permutation_gauge.py
```

Key policies.

1. Normalize read vectors.
2. Normalize write vectors where appropriate.
3. Absorb scale into amplitude.
4. Canonicalize signs.
5. Sort atoms by stable usage or norm keys for reporting.
6. Report residual gauge degrees that cannot be eliminated.

Dependencies. Can work in parallel with all benchmark subagents.

7.7 Subagent F: interaction protocols

Mission. Implement block-wise, shared-core-conditioned, and full-adapter projected interaction estimates.

Deliverables.

```
relaleap/slt/llc_interactions.py
relaleap/slt/block_protocols.py
tests/test_interaction_independent.py
tests/test_interaction_redundant.py
tests/test_interaction_synergistic.py
tests/test_full_adapter_projection_disagreement_gate.py
```

Outputs.

```
lambda_A
lambda_B
lambda_AB
I_lambda_blockwise_fixed_rest
I_lambda_with_shared_core_sampled
I_lambda_full_adapter_projected
interaction_status
```

Dependencies. Requires Subagent A for actual sampling. Can begin with mock estimators and analytic fixtures.

7.8 Subagent G: RelaLeap-shaped benchmark suite

Mission. Implement synthetic benchmark families that imitate the actual residual-column architecture.

Deliverables.

```
relaleap/slt/releap_benchmarks.py
relaleap/slt/releap_ground_truth.py
tests/test_rank_one_atom_benchmark.py
tests/test_dead_column_benchmark.py
tests/test_shared_core_benchmark.py
tests/test_mediator_benchmark.py
tests/test_router_collapse_benchmark.py
tests/test_low_rank_trap_benchmark.py
tests/test_oblique_dictionary_benchmark.py
```

Ground-truth metadata.

```
true_support
true_columns
true_basis
true_shared_core
true_mediators
true_interactions
known_regime
expected_winner
expected_I_lambda_signs
```

Dependencies. Can run in parallel once the shared benchmark interface is defined. It should not wait for the final sampler; mock analytic outputs can be used for initial tests.

7.9 Subagent H: estimator-specific nulls

Mission. Implement null distributions for SLT estimates and interaction claims.

Deliverables.

```
relaleap/slt/nulls.py
relaleap/slt/null_reports.py
tests/test_null_perturbs_declared_dependency.py
tests/test_router_support_shuffle_null.py
tests/test_shared_core_shuffle_null.py
tests/test_gauge_randomization_null.py
```

Nulls.

1. Target shuffle.
2. Residual shuffle.
3. Block-assignment shuffle.
4. Router-support shuffle.
5. Shared-core shuffle.
6. Gauge randomization.

7. Mechanism-preserving control.

8. Mechanism-violating control.

Dependencies. Parallel with Subagent G. Needs the arm dependency contract from the broader RelaLeap harness.

7.10 Subagent I: minibatch-noise and preconditioning diagnostics

Mission. Ensure that minibatch SGLD and block-wise preconditioning do not silently target the wrong posterior.

Deliverables.

```
relaleap/slt/minibatch_noise.py
relaleap/slt/preconditioning.py
tests/test_full_batch_required_for_calibration.py
tests/test_minibatch_noise_report.py
tests/test_blockwise_preconditioner_recorded.py
```

Policy. Calibration runs use full-batch gradients. RelaLeap adapter runs may use minibatches only if the gradient-noise diagnostic is reported and the estimate is marked finite-sample approximate.

Dependencies. Mostly parallel with Subagent A.

7.11 Subagent J: reporting and CI gates

Mission. Make the estimator package hard to misuse. Reports should clearly separate calibrated evidence, diagnostic-only numbers, and failures.

Deliverables.

```
relaleap/slt/reports.py
relaleap/slt/ci_gates.py
relaleap/slt/decision_report.py
tests/test_report_schema_complete.py
tests/test_failed_calibration_blocks_releap_evidence.py
tests/test_negative_lambda_blocks_promotion.py
```

Report sections.

1. Estimator target and temperature convention.

2. Chain diagnostics.

3. Analytic calibration results.

4. Sample-size sweep.

5. Gauge and prior sensitivity.

6. Interaction protocol results.

7. Null-normalized margins.

8. Final status: `calibrated`, `diagnostic_only`, or `failed`.

Dependencies. Can start early with placeholder data. Integrates all other subagent outputs.

7.12 Subagent K: integration orchestrator

Mission. Maintain the run manifest, merge branches, prevent interface drift, and run end-to-end smoke and calibration suites.

Deliverables.

```
relaleap/experiments/slt_estimator_validation.py
results/slt_estimator_validation/<run_id>/manifest.json
results/slt_estimator_validation/<run_id>/report.md
scripts/run_slt_validation_suite.sh
```

Responsibilities.

1. Freeze shared interfaces before other agents branch too far.
2. Provide mock objects for agents who do not need the full sampler.
3. Run nightly integration tests.
4. Maintain a compatibility matrix of benchmark, estimator, prior, and gauge policy.
5. Mark the package unusable for RelaLeap scientific decisions until all mandatory gates pass.

7.13 Subagent L: adversarial reviewer

Mission. Try to break the validation package. This subagent should not add new features; it should find ways the estimator can pass tests while still being wrong.

Checks.

1. Temperature convention sabotage.
2. Silent total-vs-mean loss mismatch.
3. Frozen-parameter sampling disguised as live-parameter sampling.
4. Lambda clipping or suppression of negative results.
5. Nulls that do not perturb the variables used by an arm.
6. Gauge transforms that change estimates.
7. Prior-scale flips in arm ranking.
8. Block-wise interaction signs contradicted by full-adapter runs.

8 Parallel execution schedule

8.1 Wave 0: interface freeze, one to two days

Tasks:

1. Define `EnergyModel`, `CalibrationBenchmark`, `EstimatorResult`, and report schema.
2. Create mock estimator outputs for downstream agents.
3. Create repository branches and CI skeleton.

Only Subagent K and a light contribution from Subagent A are blocking here.

8.2 Wave 1: independent implementation, three to five days

Run in parallel:

1. Subagent A implements SGLD/WBIC core.
2. Subagent B implements analytic benchmarks.
3. Subagent D implements priors and MAP refinement.
4. Subagent E implements gauge policies.
5. Subagent G implements RelaLeap-shaped benchmarks.
6. Subagent H implements nulls.
7. Subagent J implements reports using mocks.
8. Subagent L starts adversarial review of interfaces and assumptions.

This wave should not wait for perfect sampler quality. Most benchmark and reporting code can be built against mock estimators and later connected to the real sampler.

8.3 Wave 2: estimator integration, three to seven days

Tasks:

1. Connect analytic benchmarks to real SGLD/WBIC.
2. Run exact Gaussian posterior tests.
3. Run regular and rank-deficient linear calibration.
4. Run product singularity and composition benchmarks.
5. Fix scaling, prior, and sampler bugs before touching RelaLeap-shaped benchmarks.

The rule is: do not debug on RelaLeap outcomes. Debug on analytic targets.

8.4 Wave 3: interaction and sample-size validation, five to ten days

Tasks:

1. Run sample-size sweeps.
2. Fit $\Delta E_n = a \log n + b \log\log n + c$.
3. Run block-wise and joint I_λ protocols.
4. Run shared-core-conditioned and full-adapter projected checks on small benchmarks.
5. Run prior-sensitivity sweeps.

This wave can be compute-heavy but is embarrassingly parallel across seeds, benchmarks, priors, and n values.

8.5 Wave 4: RelaLeap-shaped validation, five to ten days

Tasks:

1. Run rank-one atom, dead-column, shared-core, mediator, router-collapse, low-rank-trap, and oblique-dictionary benchmarks.
2. Confirm expected winners and expected interaction signs.
3. Confirm that low-rank traps block columnar interpretation.
4. Confirm that dead columns and router collapse do not appear as false positive modularity.

Only after this wave passes should the estimator be attached to real RelaLeap residual arms.

8.6 Wave 5: real RelaLeap adapter diagnostics

Tasks:

1. Sample actual trained residual-arm parameter blocks.
2. Compare flat, SVD/low-rank, nonorthogonal dictionary, CE-gradient, Fisher/GN, and rank-one-column arms.
3. Keep prediction/reconstruction metrics, causal metrics, and SLT evidence panels separate.
4. Use SLT evidence only among candidates that already pass matched controls and causal audits.

9 Why parallel subagents help

The work naturally decomposes because the estimator core, analytic benchmark registry, gauge handling, prior sensitivity, nulls, sample-size sweeps, RelaLeap-shaped benchmarks, and reports have clear interfaces. A sequential single-agent implementation would tend to overfit early tests and would delay adversarial review until too late. Parallel subagents make three things better:

1. **Faster wall-clock progress.** Benchmark families and diagnostic reports can be written before the sampler is perfect.
2. **Less confirmation bias.** An adversarial reviewer and independent benchmark agents reduce the risk that the estimator is tuned to pass one favored case.
3. **Cleaner interfaces.** Parallel work forces the project to define explicit contracts for energy models, parameter blocks, priors, gauges, and reports.

The main risk of parallelization is interface drift. This is why Subagent K is important: one orchestrator must own the manifest, common dataclasses, CI gates, and merge discipline.

10 Final acceptance criteria

The SLT estimator package is acceptable for RelaLeap scientific diagnostics only when all of the following are true:

1. Exact Gaussian posterior tests pass.
2. Temperature-convention negative tests pass.
3. Regular and rank-deficient linear calibration pass.
4. Product singularity, redundant ridge, independent composition, redundant composition, and synergistic composition pass.
5. Sample-size slope fits are stable enough to distinguish $\log n$ effects from constants.
6. Gauge transformations do not materially alter functional LLC estimates.
7. Negative $\hat{\lambda}$ values are never clipped and correctly trigger failure or diagnostic-only status.
8. Prior sensitivity is reported and does not silently flip conclusions.
9. Estimator-specific nulls are dependency-aware and effective.
10. RelaLeap-shaped benchmarks recover known regimes and known winners.
11. Block-wise interaction claims are checked against shared-core-conditioned and full-adapter protocols.
12. Reports clearly label every number as calibrated evidence, diagnostic-only, or failed.

Only after these conditions hold should SLT estimates be used in the RelaLeap pregate. Even then, they should not replace causal audits. The correct use of SLT is to ask:

> Among residual factorizations that already match dense/low-rank controls and pass causal-modularity gates, which one has the best calibrated local-evidence geometry?

That is the operational bridge from SLT to RelaLeap.

References and project notes

1. RelaLeap WBIC/SGLD/LLC Estimator Implementation Guide, technical implementation note, 2026-07-03.
2. RelaLeap SLT Seven-Arm Pregate: Reproducibility and Diagnostic Report, OpenClaw / ZeroBot, 2026-07-02.
3. RelaLeap: Sparse Columnar Residual Adaptation of Frozen Sequence Models, Benjamin Goertzel, 2026-07-02.
4. Weakness as Local Evidence: A Systems Theory for Degeneracy, Refinement DAGs, and Continual Adaptation, with Transformers as a Case Study, Benjamin Goertzel, 2025-12-06.

5. Singular Learning Theory for SubRep: Goal-Subgoal LLC Relations in Hierarchical Planning with Functional Coupling and Shared Representations, Benjamin Goertzel, 2025-12-10.

6. Columnar Residual Networks on Top of Frozen LLMs: Weakness-Guided Causal Modular Adaptation Using Sparse Rank-One Adapter Atoms, Benjamin Goertzel and GPT-5.5-Pro, 2026-05-20.

7. Hierarchical Bayesian Causal Modular Learning: A Two-Level Columnar Architecture for Continual Learning, Benjamin Goertzel, Charlie Derr, and Yohannes Taye.